

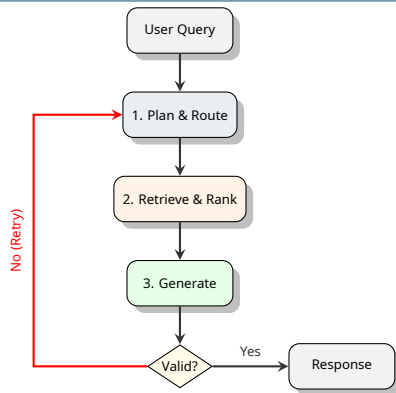
RAG with LangGraph Cheat Sheet

Building Agentic Retrieval Augmented Generation Systems

1. DEFINITION

RAG (Retrieval Augmented Generation): Enhancing LLM responses by retrieving relevant external data before generating an answer. **LangGraph Addition:** LangGraph adds stateful orchestration, loops, and conditional logic, turning a static RAG chain into an adaptive "Agentic RAG" system that can self-correct, re-query, or verify answers.

2. HIGH LEVEL ARCHITECTURE FLOW



- **Flow:** User Query → Plan Search → Retrieve Docs → Rerank Relevance → Generate Answer → Critique/Verify → Final Response.

3. CORE COMPONENTS

- **LangGraph Orchestrator:** The control engine managing the flow of data between components.
- **Nodes & Edges:** Nodes are functions (actions); Edges define the control flow (conditional or fixed).
- **State Object:** A shared dictionary (TypedDict) passing context (Query, Docs, History) between nodes.
- **Retriever Node:** Fetches relevant

chunks from the Vector DB.

- **Generator Node:** The LLM that synthesizes the answer using retrieved context.
- **Critic Node:** Evaluates answer quality or retrieval relevance (Hallucination check).
- **Memory Node:** Persists conversation history across multiple turns (Checkpoint).

4. DATA INGESTION PIPELINE

- **Document Loading:** Connectors for PDF, HTML, SQL, etc.
- **Chunking Strategies:**
 - **Fixed Size:** e.g., 500 characters with overlap. Simple but breaks semantic meaning.
 - **Recursive:** Splits by paragraphs, then sentences. Preserves structure.
 - **Semantic:** Splits based on embedding similarity changes (topic shifts).
- **Embedding Generation:** Converting text to vectors (OpenAI, HuggingFace).
- **Indexing:** Storing vectors in DB (Pinecone, Chroma, FAISS) for ANN search.

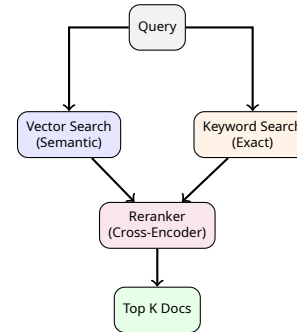
Implementation (Chunking):

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000, chunk_overlap=200
)
splits = text_splitter.split_text(text)
```

5. RETRIEVAL IMPLEMENTATION

Hybrid Search Pipeline



- **Vector Search:** Finds concepts/meaning (Cosine Similarity). Good for vague queries.
- **Hybrid Search:** Combines Vector + Keyword (BM25). Handles specific acronyms/names better.
- **Filtering:** Pre-filtering by metadata (Date, Author) before search.
- **Reranking:** Using a high-precision model (Cross-Encoder) to re-sort the top K results.

Implementation (Rerank Node):

```
def rerank_node(state):
    # Re-order docs by relevance score
    ranked = reranker.rank(
        state["question"], state["documents"]
    )
    return {"documents": ranked[:3]} # Top 3
```

6. LANGGRAPH GRAPH DESIGN

Defining State & Graph:

```
from typing import TypedDict, List
from langgraph.graph import StateGraph, END

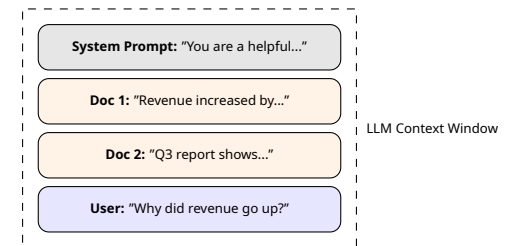
class AgentState(TypedDict):
    question: str
    documents: List[str]
    generation: str

workflow = StateGraph(AgentState)
workflow.add_node("retrieve", retrieve_node)
workflow.add_node("generate", generate_node)
workflow.add_edge("retrieve", "generate")
workflow.add_edge("generate", END)
app = workflow.compile()
```

- **Defining Nodes:** Python functions taking state as input and returning state updates.

- **Defining Edges:** Linking nodes. `workflow.add_edge("a", "b")`.
- **Conditional Routing:** Logic to decide next step (e.g., if docs_found goto generate, else rewrite_query).
- **Loops & Retries:** Cycles in the graph (e.g., Critic rejects → Generator retries) to improve quality.

7. PROMPT & CONTEXT MANAGEMENT



- **Context Window:** Fitting retrieved docs into LLM limits. Truncation or summarization may be needed.
- **Source Injection:** Clearly formatting docs in the prompt: Context: [ID] Text: ...
- **Citation Control:** Instructing LLM to reference specific Document IDs in the final answer.

8. END TO END EXAMPLE FLOW

Scenario: "Explain the Q3 revenue drop."

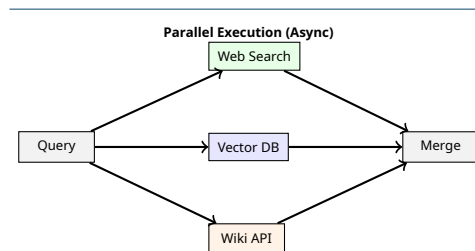
1. **User:** Asks question. Graph initializes state.
2. **Router:** Classifies intent as "Financial Analysis". Routes to Finance Subgraph.
3. **Retrieve:** Fetches Q3 Reports + Email logs.

4. **Rerank:** Prioritizes "Q3 Summary PDF" over generic emails.
5. **Generate:** LLM drafts answer citing the PDF.
6. **Verify:** Critic checks if numbers match the PDF text.
7. **Return:** Validated response sent to user.

9. PERFORMANCE METRICS

- **Answer Accuracy:** Does the answer match ground truth? (Human eval or LLM-as-judge).
- **Source Grounding Rate:** % of statements supported by retrieved docs (Faithfulness).
- **Latency:** Time to first token (TTFT) and total time (p50, p95, p99).
- **Retrieval Recall:** Is the correct document appearing in the top K?
- **Token Usage:** Input vs Output tokens (cost driver).
- **Cost per Query:** Total infrastructure + API cost.

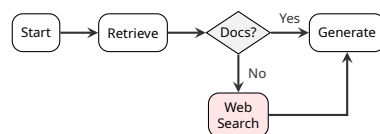
10. SCALING AND OPTIMIZATION



- **Caching:** Cache embeddings and LLM responses (Redis) for identical queries.
- **Batching Retrieval:** Fetching docs for multiple user queries in one DB call.
- **Index Tuning:** Optimizing HNSW parameters (ef_construction, M) for speed vs recall.
- **Parallel Nodes:** Running web search and DB search concurrently in LangGraph (async).

11. RELIABILITY AND SAFETY

Reliability Graph Pattern



Implementation (Conditional Edge):

```

def check_hallucination(state):
    # LLM-as-Judge
    score = grader.invoke(state)
    if score == "yes":
        return "generate" # Loop back
    return END # Finish
  
```

- **Fallback Retrieval:** If Vector DB finds nothing, fall back to Web Search or Keyword Search.
- **Hallucination Detection:** A separate "Grader" agent checking answer vs context.
- **Timeouts:** Setting strict limits on graph node execution time.
- **Error Handling:** Try/Except blocks within nodes to return graceful error messages.

12. COST OPTIMIZATION

- **Smaller Embedding Models:** Use quantized or smaller models for vectorization if accuracy permits.
- **Compression:** Summarize retrieved docs using a small LLM before passing to the main generator.
- **Selective Retrieval:** Only run retrieval if the query actually needs external data (Router Node).

13. COMMON FAILURE POINTS

- **Poor Chunking:** Splitting tables or code blocks mid-stream effectively destroys context.
- **Low Recall:** The answer is in the DB, but the search didn't find it (bad embeddings or search params).
- **Context Overflow:** Retrieving too many docs pushes useful info out of the LLM's attention window.
- **Over Retrieval:** Fetching irrelevant docs that confuse the LLM ("Lost in the Middle" phenomenon).

14. SUMMARY

Goal: Create an intelligent, self-correcting knowledge retrieval system.

- **Core:** LangGraph loops enable "Reasoning" over static retrieval.
- **Key:** Good retrieval (hybrid/rerank) is the foundation of good generation.
- **Outcome:** Trustworthy, cited, and domain-specific AI answers.